

Task Distribution: Two Fullstack Developers

Overview

This document distributes development tasks between **Developer 1** and **Developer 2** to enable parallel development with minimal dependencies and conflicts.

AI Tasks: All AI-related tasks (Epic 4) are excluded and assigned to the AI Engineer separately.

Development Strategy

Core Principle

Split by **domain boundaries** with well-defined interfaces:

- **Developer 1:** User & Order Domain (User-facing features, order lifecycle)
- **Developer 2:** Financial & Compliance Domain (Escrow, payments, Shariah)

Integration Points (Define First)

Before development starts, both developers must agree on:

1. **Order Model/Schema** - Database model and API schema for orders
 2. **Order Status Enum** - Standard status values (CREATED, FUNDED, IN_TRANSIT, DELIVERED, SETTLED, etc.)
 3. **User Model/Schema** - User/Account structure with roles (MSME, BUYER)
 4. **API Contract** - REST endpoints for cross-domain communication
-

Developer 1: User & Order Management

Focus: Authentication, user management, order creation, delivery management, order viewing

Epic 1: Identity & Access Management

Feature 1.1: User Registration and Authentication

Backend Tasks:

- ☐ User Story 1.1.1: MSME Registration API endpoint
 - POST `/api/v1/auth/register/msme`
 - Create MSME account model
 - Validate business details
 - Set status to PENDING_VERIFICATION
 - Return credentials
- ☐ User Story 1.1.2: Buyer Registration API endpoint

- POST `/api/v1/auth/register/buyer`
 - Create buyer account model
 - Validate buyer details
 - Set status to ACTIVE
 - Return credentials
- ☐ User Story 1.1.3: User Login API endpoint
 - POST `/api/v1/auth/login`
 - Validate credentials (username/phone + password)
 - Generate JWT token with role and permissions
 - Token expiration (24 hours)
 - Return JWT token
- ☐ User Story 1.1.4: Role-Based Access Control (RBAC)
 - Create RBAC middleware/decorator
 - Implement role verification logic
 - Return 403 for insufficient permissions
 - Integrate with FastAPI dependency injection

Frontend Tasks:

- ☐ MSME Registration Page
 - Form with: business name, registration number, phone, password
 - API integration with registration endpoint
 - Success/error handling
 - Redirect to login on success
- ☐ Buyer Registration Page
 - Form with: name, phone, address, password
 - API integration with registration endpoint
 - Success/error handling
 - Redirect to login on success
- ☐ Login Page
 - Form with: username/phone, password
 - API integration with login endpoint
 - Store JWT token (localStorage/sessionStorage)
 - Role-based redirect after login
 - Error handling
- ☐ Protected Route Wrapper
 - JWT token validation
 - Role-based route protection
 - Redirect to login if unauthenticated
 - Redirect based on user role

Feature 1.2: Business Verification

Backend Tasks:

- ☐ User Story 1.2.1: MSME Business Document Upload API
 - POST `/api/v1/msme/documents/upload`
 - File upload handling (store file reference)
 - Update MSME status to VERIFICATION_IN_PROGRESS
 - Store document references

Frontend Tasks:

- ☐ Business Document Upload Page/Component
 - File upload component
 - Display uploaded documents
 - Status indicator (PENDING_VERIFICATION, VERIFICATION_IN_PROGRESS)
 - API integration
-

Epic 2: Orders & Trade Orchestration

Feature 2.1: Order Creation

Backend Tasks:

- ☐ User Story 2.1.1: Create Escrow Order API
 - POST `/api/v1/orders`
 - Create order model with status CREATED
 - Validate buyer exists and is ACTIVE
 - Store order details (buyer ID, product description, amount, delivery terms)
 - Return order ID and state
- ☐ User Story 2.1.2: Order Validation
 - Input validation middleware/schema
 - Validate: amount > 0, buyer exists, valid delivery date
 - Return validation errors with specific field issues
 - Prevent invalid order creation

Frontend Tasks:

- ☐ Create Order Page (MSME Dashboard)
 - Form with: buyer selection, product description, amount, delivery terms
 - Buyer search/selection component
 - Amount input with validation
 - Delivery date picker
 - API integration
 - Success/error handling
 - Redirect to order details after creation

Feature 2.2: Order State Management

Backend Tasks:

- ☐ User Story 2.2.1: View Order Status API
 - GET `/api/v1/orders/{order_id}`
 - Return order details and current status
 - Check user authorization (MSME owner or buyer)
 - Return: status, amount, parties, dates, current state
- ☐ User Story 2.2.2: List Orders API
 - GET `/api/v1/orders` (with filters)
 - Filter by status, date range
 - Pagination support
 - Return orders owned by user (as MSME) or where user is buyer
 - Sort by creation date descending

Frontend Tasks:

- ☐ Order Details Page
 - Display full order information
 - Status badge/timeline component
 - Order details (amount, parties, dates)
 - API integration
 - Error handling
- ☐ Order List Page
 - List view with pagination
 - Filter by status
 - Filter by date range
 - Order cards with summary (ID, status, amount, date)
 - API integration
 - Link to order details

Feature 2.3: Delivery Management

Backend Tasks:

- ☐ User Story 2.3.1: Mark Order as Delivered API
 - POST `/api/v1/orders/{order_id}/delivery/confirm`
 - Update order status to DELIVERED (if status is IN_TRANSIT or FUNDED)
 - Record delivery timestamp and confirmation details
 - Trigger delivery verification workflow (event/notification to Developer 2's module)
- ☐ User Story 2.3.2: Delivery Dispute Initiation API

- POST `/api/v1/orders/{order_id}/disputes`
- Create dispute record linked to order
- Prevent automatic payment release (set flag)
- Notify MSME of dispute

Frontend Tasks:

- ☐ Delivery Confirmation Page/Component (Buyer)
 - Form to confirm delivery
 - Input: delivery date, notes
 - API integration
 - Success/error handling
 - ☐ Delivery Dispute Page/Component (Buyer)
 - Form with: dispute reason, description
 - API integration
 - Success/error handling
-

Developer 1 Summary

Backend Modules to Create:

- `app/models/user.py` - User, MSME, Buyer models
- `app/models/order.py` - Order model (define status enum for Developer 2)
- `app/api/v1/auth.py` - Authentication endpoints
- `app/api/v1/orders.py` - Order management endpoints
- `app/core/auth.py` - JWT, RBAC middleware
- `app/services/user_service.py` - User management logic
- `app/services/order_service.py` - Order management logic

Frontend Pages/Components:

- Authentication pages (login, register MSME, register buyer)
- MSME dashboard (create order, view orders)
- Buyer dashboard (view orders, confirm delivery, disputes)
- Order list and details pages
- Protected route wrapper

Key Deliverables:

- ☒ User authentication system (login, registration)
 - ☒ Order CRUD operations
 - ☒ Order status management
 - ☒ Delivery confirmation workflow
 - ☒ RBAC implementation
-

💰 Developer 2: Financial & Compliance

Focus: Escrow funding, payment release, Shariah compliance, order orchestration, audit

Epic 3: Escrow & Settlement

Feature 3.1: Escrow Funding

Backend Tasks:

- ☐ User Story 3.1.1: Fund Escrow Account API
 - POST `/api/v1/escrow/{order_id}/fund`
 - Validate order exists and status is CREATED
 - Validate funding amount matches order amount
 - Create escrow balance record (simulated)
 - Update order status to FUNDED
 - Record funding timestamp
- ☐ User Story 3.1.2: Escrow Balance Validation
 - Create escrow balance model
 - Validation service to check balance $\geq$ order amount
 - Integration with order progression logic
 - Return clear error if insufficient balance

Frontend Tasks:

- ☐ Fund Escrow Page/Component (Buyer)
 - Display order details and amount
 - Confirm funding action
 - API integration with funding endpoint
 - Success/error handling
 - Redirect to order details
- ☐ Escrow Balance Display Component
 - Show current escrow balance for order
 - Status indicator (funded/unfunded)
 - API integration

Feature 3.2: Payment Release

Backend Tasks:

- ☐ User Story 3.2.1: Release Payment to MSME API
 - POST `/api/v1/escrow/{order_id}/release` (automated trigger)
 - Validate order status is DELIVERED
 - Check all release conditions (Shariah COMPLIANT, AI APPROVED - via interface, delivery confirmed)
 - Update order status to SETTLED

- Record payment release transaction (simulated)
 - Update MSME account balance (simulated)
 - Record settlement timestamp
 - Notify MSME (event/notification)
- ☐ User Story 3.2.2: Refund to Buyer API
 - POST `/api/v1/escrow/{order_id}/refund`
 - Validate refund conditions (cancelled, Shariah violation, AI rejection)
 - Update order status to REFUNDED or CANCELLED
 - Record refund transaction (simulated)
 - Update buyer account balance (simulated)
 - Notify buyer (event/notification)

Frontend Tasks:

- ☐ Payment Release Status Component
 - Display payment release status
 - Show release conditions (Shariah, AI, delivery)
 - Status timeline
 - API integration
- ☐ Refund Status Component
 - Display refund status and reason
 - Refund amount display
 - API integration

Feature 3.3: Escrow State Tracking

Backend Tasks:

- ☐ User Story 3.3.1: Escrow Transaction History API
 - GET `/api/v1/escrow/{order_id}/transactions`
 - Return chronological list of transactions (funding, release, refund)
 - Include: type, amount, timestamp, status
 - Sort by timestamp descending

Frontend Tasks:

- ☐ Transaction History Component
 - Display transaction list
 - Filter by transaction type
 - Timeline view
 - API integration

Epic 5: Shariah Compliance

Feature 5.1: Shariah Rule Validation

Backend Tasks:

- ☐ User Story 5.1.1: Validate Order Against Shariah Rules
 - POST `/api/v1/shariah/validate/{order_id}` or auto-trigger on order creation/modification
 - Create Shariah validation service
 - Check for prohibited elements (riba, gambling, haram goods/services)
 - Validate transaction structure (no interest, proper escrow)
 - Return compliance status (COMPLIANT, NON_COMPLIANT, REQUIRES_REVIEW)
 - Complete within 1 second (rule-based, not ML)
- ☐ User Story 5.1.2: Shariah Violation Handling
 - Handle NON_COMPLIANT status
 - Prevent order progression (block funding or release)
 - Record violation details (rule violated, reason)
 - Trigger notifications to MSME and Buyer
 - Trigger refund workflow if funds in escrow
 - Update order status to indicate compliance issue

Frontend Tasks:

- ☐ Shariah Compliance Status Component
 - Display compliance status badge
 - Show validation timestamp
 - Display violation reasons if non-compliant
 - API integration
- ☐ Shariah Compliance Card
 - Detailed compliance information
 - Rules checked
 - Compliance certificate link (if applicable)

Feature 5.2: Shariah Compliance Reporting**Backend Tasks:**

- ☐ User Story 5.2.1: Shariah Compliance Status API
 - GET `/api/v1/shariah/{order_id}/status`
 - Return compliance status
 - Return validation timestamp
 - Return reason if non-compliant
- ☐ User Story 5.2.2: Shariah Compliance Certificate API
 - GET `/api/v1/shariah/{order_id}/certificate`
 - Generate certificate document (PDF or JSON)

- Include: order details, compliance verification timestamp, Shariah principles validated, transaction summary
- Digital signature/verification (simplified for MVP)

Frontend Tasks:

- ☐ Compliance Certificate View/Download
 - Display certificate content
 - Download button (PDF)
 - API integration
-

Epic 6: System Integration & Orchestration**Feature 6.1: End-to-End Order Workflow****Backend Tasks:**

- ☐ User Story 6.1.1: Complete Order Lifecycle Orchestration
 - Create orchestration service
 - Enforce state transitions:
 - CREATED → FUNDED (after escrow funding)
 - FUNDED → IN_TRANSIT (after Shariah compliance + AI approval)
 - IN_TRANSIT → DELIVERED (after buyer confirms delivery)
 - DELIVERED → SETTLED (after all checks, payment released)
 - Reject invalid state transitions
 - Log state changes with timestamps
- ☐ User Story 6.1.2: Multi-Condition Payment Release Gate
 - Create payment release service
 - Verify all conditions before release:
 - Shariah compliance status is COMPLIANT
 - AI approval status is APPROVED (call AI service interface)
 - Delivery confirmation exists
 - Escrow balance is sufficient
 - Release payment if all conditions met
 - Block payment with reason logged if any condition fails
 - Notify relevant parties

Frontend Tasks:

- ☐ Order Lifecycle Timeline Component
 - Visual timeline of order states
 - Current status indicator
 - Transition history
 - API integration

Feature 6.2: Error Handling & Notifications

Backend Tasks:

- ☐ User Story 6.2.1: System Error Handling
 - Global error handler middleware
 - Log errors with context (user, order, module, timestamp)
 - Return appropriate HTTP status codes (400, 403, 404, 500)
 - Return user-friendly error messages
 - Ensure transaction state consistency
- ☐ User Story 6.2.2: Order Status Notifications
 - Create notification service
 - Send notifications on status changes (FUNDED, DELIVERED, SETTLED, CANCELLED)
 - Include: order ID, new status, relevant context
 - Log notifications for audit
 - Integration with frontend (WebSocket or polling)

Frontend Tasks:

- ☐ Notification System
 - Display notifications (toast, banner, or notification center)
 - Real-time updates (WebSocket or polling)
 - Mark as read functionality
 - API integration

Feature 6.3: Audit & Logging**Backend Tasks:**

- ☐ User Story 6.3.1: Transaction Audit Log
 - Create audit log model
 - Log critical operations:
 - Order creation, funding, release
 - AI decisions (via interface)
 - Shariah validation
 - Include: operation type, timestamp, user/actor, entity affected, result, IP address
 - Make logs searchable

Frontend Tasks:

- ☐ Audit Log View (Admin/Dashboard)
 - Display audit logs with filters
 - Search functionality
 - Export functionality (optional)
 - API integration

Developer 2 Summary**Backend Modules to Create:**

- `app/models/escrow.py` - Escrow balance, transactions models
- `app/models/shariah.py` - Shariah validation results model
- `app/models/audit.py` - Audit log model
- `app/api/v1/escrow.py` - Escrow endpoints
- `app/api/v1/shariah.py` - Shariah endpoints
- `app/services/escrow_service.py` - Escrow management logic
- `app/services/shariah_service.py` - Shariah validation logic
- `app/services/orchestration_service.py` - Order lifecycle orchestration
- `app/services/notification_service.py` - Notification logic
- `app/services/audit_service.py` - Audit logging logic

Frontend Pages/Components:

- Escrow funding pages
- Payment release status components
- Transaction history views
- Shariah compliance components
- Order lifecycle timeline
- Notification system
- Audit log views

Key Deliverables:

- ☒ Escrow funding and settlement system
- ☒ Payment release automation
- ☒ Shariah compliance validation
- ☒ Order lifecycle orchestration
- ☒ Notification system
- ☒ Audit logging

Integration & Coordination

Required Shared Contracts (Define Before Development Starts)

1. Order Model/Schema (Critical - Define First)

```
# Shared between both developers
OrderStatus = Enum:
    CREATED
    FUNDED
    IN_TRANSIT
    DELIVERED
    SETTLED
    CANCELLED
    REFUNDED

Order Model:
    - id
```

- msme_id (foreign key)
- buyer_id (foreign key)
- amount
- status (OrderStatus)
- product_description
- delivery_terms
- created_at
- updated_at
- shariah_status (COMPLIANT/NON_COMPLIANT/REQUIRES_REVIEW) # Developer 2
- ai_approval_status (APPROVED/REJECTED/PENDING_REVIEW) # AI Engineer
- delivery_confirmed_at # Developer 1
- escrow_funded_at # Developer 2

2. User Model/Schema (Developer 1 creates, Developer 2 references)

User Model:

- id
- username/phone
- password_hash
- role (MSME/BUYER)
- status (ACTIVE/PENDING_VERIFICATION/etc)
- created_at

3. API Contracts (REST Endpoints)

Developer 1 provides:

- `GET /api/v1/orders/{order_id}` - Developer 2 needs this
- `PUT /api/v1/orders/{order_id}/status` - For status updates (orchestration)

Developer 2 provides:

- `GET /api/v1/escrow/{order_id}/balance` - Developer 1 may need this
- `GET /api/v1/shariah/{order_id}/status` - Developer 1 may need this

AI Engineer provides (interface only):

- `POST /api/v1/ai/evaluate/{order_id}` - Returns AI approval status
- `GET /api/v1/ai/explanation/{order_id}` - Returns AI decision explanation

4. Events/Notifications (Optional but Recommended)

Define event structure for cross-module communication:

- `OrderStatusChanged` event
- `PaymentReleaseTriggered` event
- `ShariahValidationCompleted` event

⊗ Dependencies & Potential Conflicts

How to Avoid Conflicts

1. Database Models:

- Developer 1 creates **Order** model first
- Developer 2 adds fields to **Order** model (shariah_status, escrow fields) via migrations
- Use feature branches and coordinate merges

2. API Endpoints:

- Use separate route files (**orders.py** vs **escrow.py**)
- No overlapping endpoints

3. Frontend:

- Separate pages/components
- Developer 1: auth/, orders/, delivery/
- Developer 2: escrow/, shariah/, transactions/
- Shared components in **components/ui/**

4. Status Updates:

- Developer 1 updates status to DELIVERED
- Developer 2 updates status to FUNDED, SETTLED, REFUNDED
- Use orchestration service to coordinate transitions

Development Order (Recommended Sequence)

Phase 1: Foundation (Week 1, Days 1-2)

Both developers:

- Define shared models (Order, User) together
- Set up database schema
- Agree on API contracts

Phase 2: Core Features (Week 1, Days 3-5)

Developer 1:

- User registration/login (backend + frontend)
- Order creation (backend + frontend)
- Order viewing (backend + frontend)

Developer 2:

- Escrow funding (backend + frontend)
- Shariah validation service (backend)
- Escrow balance tracking (backend)

Phase 3: Integration (Week 2, Days 1-3)

Developer 1:

- Delivery confirmation (backend + frontend)
- Order status updates

Developer 2:

- Payment release automation (backend)
- Orchestration service (backend)
- Transaction history (backend + frontend)

Phase 4: Polish (Week 2, Days 4-5)

Developer 1:

- Business verification (if time permits)
- Order list with filters

Developer 2:

- Notification system (backend + frontend)
- Audit logging (backend + frontend)
- Shariah compliance UI

☒ Checklist Before Starting

- ☐ Both developers review and agree on shared models
- ☐ Database schema defined and migrated
- ☐ API contracts documented (OpenAPI/Swagger)
- ☐ Git branching strategy agreed (feature branches, main branch protection)
- ☐ Code review process defined
- ☐ Development environment setup completed
- ☐ AI Engineer interface defined (for payment release gate)

Notes

- **AI Integration:** Developer 2's payment release gate will call AI service interface. AI Engineer provides mock/stub during development.
- **Testing:** Each developer writes tests for their modules. Integration tests can be added later.
- **Documentation:** Each developer documents their API endpoints as they build them.

Last Updated: [Date]

Next Review: After Phase 2 completion